

คู่มือสำหรับโปรแกรมเมอร์

การพัฒนาเว็บเซอร์วิสบนระบบฝังตัว

Embedded Web Service Development

Embedded SOAP Server

Programmer Manual

โดย

นายธเนศ ชูวิเศษวณิชย์ 43010175

นายธีรยุทธ สุทธิสุวรรณ 43010189

อาจารย์ที่ปรึกษา

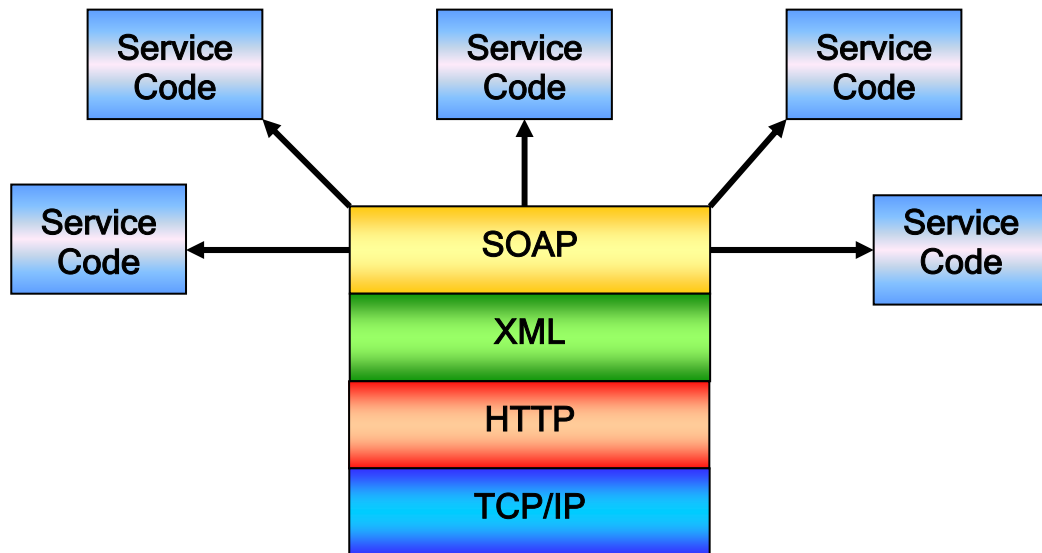
ผศ. อภิเนตร อุณาจูล

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ปีการศึกษา 2546

สถาปัตยกรรมของอาร์พีซีเซอร์วิส

สำหรับสถาปัตยกรรมของระบบให้บริการเว็บเซอร์วิสนั้น ประกอบขึ้นจากเทคโนโลยีพื้นฐานในชั้นการติดต่อสื่อสารข้อมูลแต่ละชั้น ดังนี้

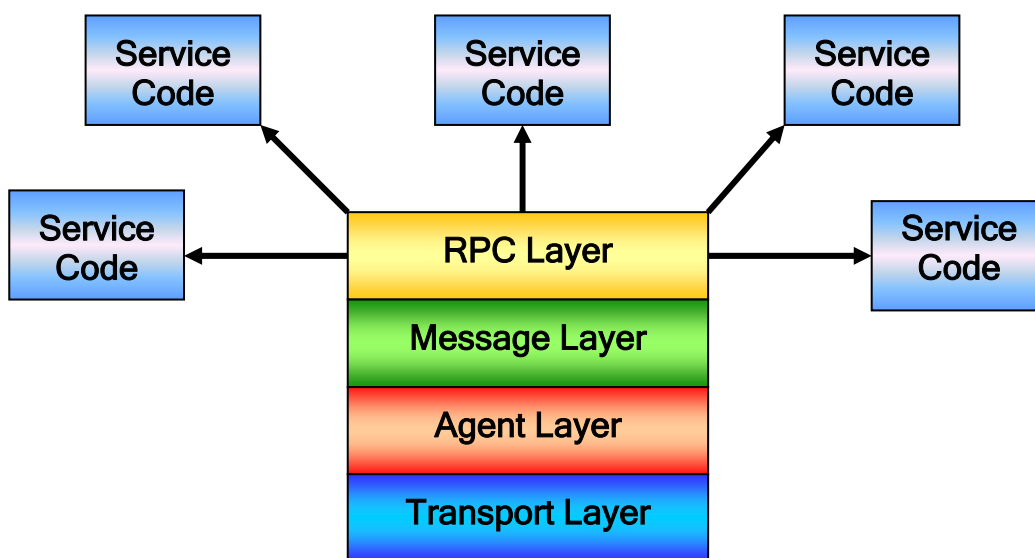


รูปภาพ 1 โครงสร้างของสถาปัตยกรรมของเว็บเซอร์วิส

1. **ชั้นทรานสปอร์ต (Transport Layer)** เป็นชั้นที่ทำหน้าที่เชื่อมต่อระดับเน็ตเวิร์ก ซึ่งทำการส่งข้อมูลระหว่างระบบคอมพิวเตอร์ที่ติดต่อกันผ่านเครือข่ายเน็ตเวิร์กซึ่งได้แก่ เครือข่ายอินเทอร์เน็ต โดยโปรโตคอลที่ใช้สำหรับการเชื่อมต่อในระดับนี้ ได้แก่ โปรโตคอลทีซีพี
2. **ชั้นแอปพลิเคชัน (Application Layer)** เป็นชั้นที่ทำหน้าที่สื่อสารในระดับของการติดต่อสื่อสารข้อมูลเพื่อทำงานใดงานหนึ่งโดยเฉพาะ เช่น โปรโตคอลเอชทีทีพี (HTTP) ใช้ในการส่งข้อมูลในรูปแบบของเอกสารไฮเปอร์เท็กซ์ เพื่อแสดงผลเกี่ยวกับเว็บเพจ , โปรโตคอลเอฟทีพี (FTP) ใช้ในการส่งข้อมูลในรูปแบบของการรับส่งไฟล์ข้อมูล , โปรโตคอลเอสเอ็มทีพี (SMTP) ใช้ในการส่งข้อมูลในรูปแบบของการจัดส่งจดหมายอิเล็กทรอนิกส์ (Electronic mail) เป็นต้น
3. **ชั้นเอกสารในการแปลความหมาย (Message Layer)** เป็นชั้นที่ทำหน้าที่เป็นรูปแบบของเอกสารเพื่อกำหนดวิธีการแปลความหมายของข้อมูลที่ส่งถึงกัน เช่น เอกสารเอชทีเอ็มแอล เป็นเอกสารที่มีรูปแบบในการแปลความหมายเพื่อการแสดงผลผ่านหน้าจอบราวเซอร์ (Web Browser) , เอกสารเอ็กซ์เอ็มแอล เป็นเอกสารที่มีรูปแบบในการแปลความหมายที่สามารถกำหนดเองได้ ซึ่งเป็นแบบมาร์กอัปแลงกเวจ (Markup Language) โดยเฉพาะ ใช้ในการสื่อสารข้อมูลระหว่างแอปพลิเคชัน เป็นต้น
4. **ชั้นแอปพลิเคชันที่จัดการบริการ (Prosedure Call Layer)** เป็นชั้นที่ทำหน้าที่จัดการและเรียกใช้บริการต่างๆ ที่มีอยู่ในระบบในรูปแบบของการเรียกใช้ฟังก์ชัน ซึ่งทำให้ไม่ต้องขึ้นกับ

แพลตฟอร์ม ตัวอย่างเช่น ซิมเปิลโออบเจกต์แอคเซสโพรโตคอล หรือ โซบ (Simple Object Access Protocol : SOAP) ที่เป็นส่วนที่ทำหน้าที่ในการเรียกใช้บริการต่างๆ ในสถาปัตยกรรมเว็บเซอร์วิส , คอรับา (CORBA) เป็น

จากสถาปัตยกรรมเว็บเซอร์วิส ที่ประกอบไปด้วยชั้นต่างๆ ที่ทำหน้าที่เฉพาะตามแต่ละโพรโตคอล จึงสามารถสรุปให้มีความหมายในลักษณะกว้างๆ เพื่อที่จะชี้ให้เห็นถึงความสามารถของสถาปัตยกรรมการออกแบบที่สามารถครอบคลุมรูปแบบการติดต่อสื่อสารแบบอื่นๆได้



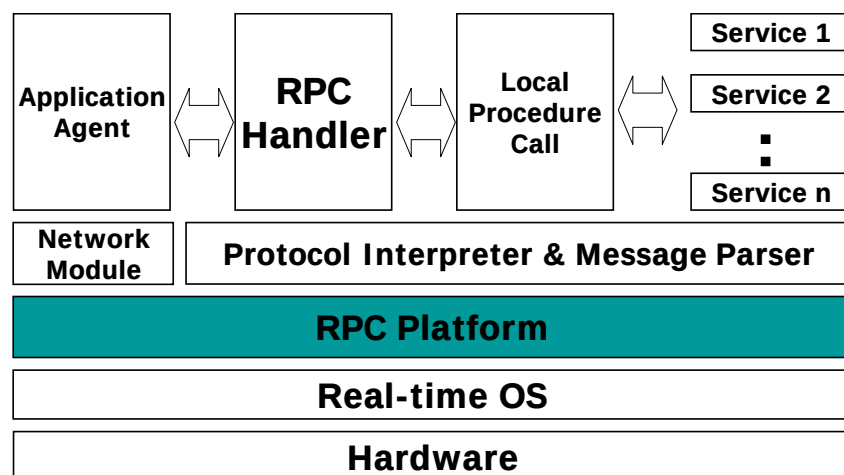
รูปภาพ 2 สถาปัตยกรรมเว็บเซอร์วิสในมุมมองของอาร์ทีซี

จากสถาปัตยกรรมดังกล่าว จะทำให้การทำงานในแต่ละชั้นนั้น สามารถทำงานได้อย่างอิสระต่อกัน ทำให้ยืดหยุ่นต่อการนำไปพัฒนาเปลี่ยนแปลงชั้นสื่อสารแต่ละชั้น ตามแต่ละมาตรฐานหรือโพรโตคอล ทำให้ไม่จำเป็นต้องพัฒนาระบบใหม่ทั้งหมด ทำให้สามารถพัฒนาหรือเปลี่ยนแปลงไปตามแต่ละมาตรฐานได้อย่างรวดเร็ว โดยยกตัวอย่างโพรโตคอลต่างๆ ในแต่ละชั้นได้ดังนี้

Application Layer	SOAP Service	Web Server	SNMP Service
Transport Layer	TCP/IP	TCP/IP	UDP/IP
Agent Layer	HTTP	HTTP	SNMP
Message Layer	XML	HTML	Text format
RPC	SOAP	CGI & File Control	Network Management

การวิเคราะห์จากมุมมองดังกล่าวนี้ เพื่อเป็นแนวคิดในการออกแบบสถาปัตยกรรมเว็บเซอร์วิสที่สามารถดัดแปลงโครงสร้างให้สามารถรองรับการพัฒนาไปสู่การให้บริการอื่นๆได้ ภายในระบบเดียวกันเป็นแบบมัลติโปรโตคอล – มัลติเซอร์วิส (Multi-Protocol & Multi-Service) เพื่อเพิ่มความสามารถในการติดต่อสื่อสารบนอุปกรณ์ฝังตัว ให้ความหลากหลายคล้ายกับบนระบบให้บริการคอมพิวเตอร์ในระบบเอนเตอร์ไพรส์ (Enterprise Server)

สถาปัตยกรรมของอาร์พีซีเซอร์วิสบนระบบปฏิบัติการแบบเรียลไทม์



รูปภาพ 3 สถาปัตยกรรมของอาร์พีซีเซอร์วิสบนระบบปฏิบัติการแบบเรียลไทม์

จากแนวคิดในการวิเคราะห์หน้าที่ของส่วนต่างๆในระบบเว็บเซอร์วิส ซึ่งสามารถแยกออกเป็น ส่วนๆซึ่งทำงานอิสระต่อกัน รวมกับแนวคิดในการพัฒนาโปรแกรมบนระบบปฏิบัติการแบบเรียลไทม์ซึ่งสามารถรองรับการทำงานในแต่ละส่วนของระบบได้อย่างอิสระ และยังสามารถทำงานหลายๆงานได้ในเวลาเดียวกันด้วยเทคนิคของการ Scheduling โดยกลไกภายในของเคอร์เนล ทำให้สามารถนำแต่ละส่วนของสถาปัตยกรรมอาร์พีซีเซอร์วิส มาสร้างขึ้นเป็นทาสก์ย่อยๆ ที่ทำงานอยู่บนเคอร์เนลเดียวกัน ดังรูป

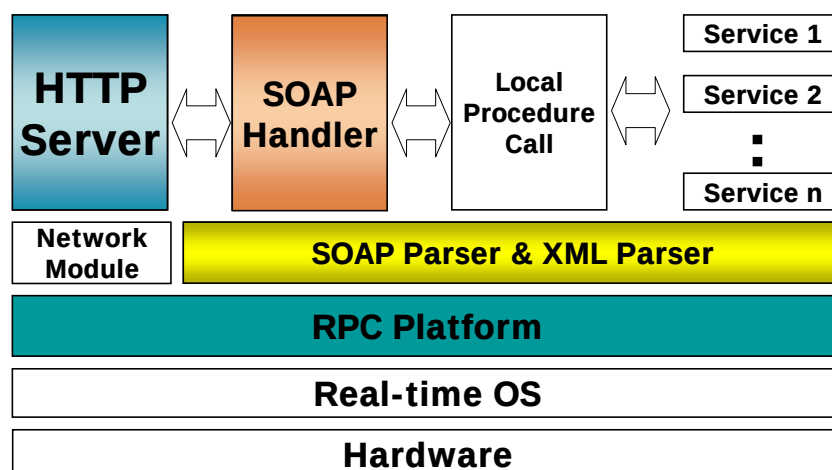
ส่วนประกอบต่างๆ สามารถอธิบายได้ดังนี้

- ฮาร์ดแวร์ (Hardware) เป็นระบบอุปกรณ์ฝังตัว ที่นำมาใช้งาน โดยเป็นแพลตฟอร์มที่พัฒนาความสามารถในด้านต่างๆ ให้ทำงานตามที่ต้องการโดยโปรแกรมและระบบปฏิบัติการ
- ไมโครเคอร์เนล (Micro Kernel) เป็นส่วนของระบบปฏิบัติการที่ควบคุมอุปกรณ์ที่เป็นฮาร์ดแวร์ โดยตรง และจัดการงานต่างๆ รวมถึงเตรียมระบบที่ช่วยให้โปรแกรมสามารถทำงานร่วมกันได้อย่างมีประสิทธิภาพ

- **เน็ตเวิร์คโมดูล (Network Module)** เป็นส่วนของโปรแกรมในระบบที่ทำหน้าที่ในการจัดการและควบคุมการติดต่อกับเครื่องคอมพิวเตอร์หรืออุปกรณ์ฝังตัวอื่นๆ ผ่านระบบเน็ตเวิร์ค โดยโปรโตคอลได้แก่ ทีซีพี/ไอพี , ยูดีพี/ไอพี เป็นต้น
- **ส่วนจัดการโปรโตคอลและอ่านเอกสาร (Protocol Interpreter & Message Parser)** เป็นยูทิลิตี้ที่ช่วยให้โปรแกรมบนระบบ สามารถจัดการกับเอกสาร หรือข้อมูลในรูปแบบของโปรโตคอลต่างๆ ที่เป็นมาตรฐานในการติดต่อขอใช้บริการระหว่างกัน ตัวอย่างส่วนจัดการโปรโตคอล เช่น โซบพาร์เซอร์ (SOAP Parser) , เอชทีทีพีพาร์เซอร์ (HTTP Parser) เป็นต้น และตัวอย่างยูทิลิตี้ที่ช่วยในการอ่านและจัดการเอกสาร เช่น เอกซ์เอ็มแอลพาร์เซอร์ (XML Parser) เป็นต้น
- **เอเจนต์ (Agent)** เป็นส่วนที่ทำหน้าที่ติดต่อสื่อสารข้อมูลจนได้ข้อมูลที่เป็นเอกสารหรือข้อมูลในรูปแบบของโปรโตคอลในการเรียกใช้บริการ ซึ่งทำงานบนส่วนของเน็ตเวิร์คโมดูลอีกทีหนึ่ง ตัวอย่างเช่น เอชทีทีพีเชิร์ฟเวอร์ , เอฟเอ็มทีพีเชิร์ฟเวอร์ เป็นต้น
- **อาร์พีซี (RPC)** เป็นส่วนที่ประสานงานเพื่อทำการติดต่อขอใช้บริการในระบบ หลังจากดึงข้อมูลในการเรียกใช้บริการโดยส่วนจัดการโปรโตคอล ตัวอย่างเช่น โซบเซอร์วิสทาสก์ (SOAP Service Task) เป็นต้น
- **บริการ (Service)** เป็นฟังก์ชันโค้ดที่เป็นบริการที่มีอยู่บนระบบ

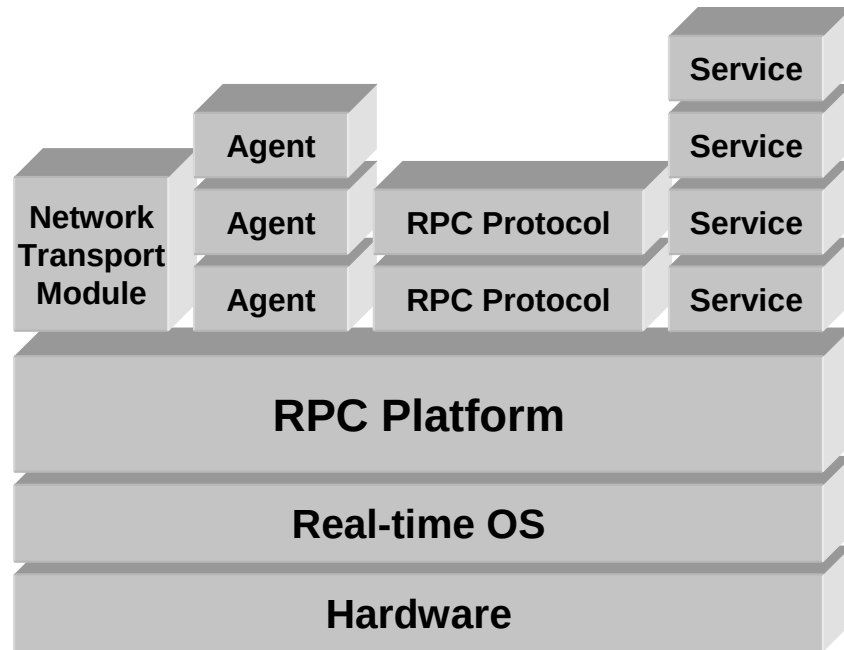
จากสถาปัตยกรรมโครงสร้างดังกล่าว สามารถย้อนกลับไปรองรับส่วนต่างๆ ของระบบเว็บเซอร์วิสได้อย่างลงตัว ตามตารางดังนี้

ส่วนของอาร์พีซีเซอร์วิส	ส่วนของเว็บเซอร์วิส	หน้าที่
Agent	HTTP Server	รองรับการติดต่อในรูปแบบของเอชทีทีพี
RPC	SOAP	แปลงเอกสารโซบ แล้วสร้างการเรียกใช้บริการ
Message Parser	XML Parser	อ่านและเขียนเอกสารในรูปแบบของเอกซ์เอ็มแอล



รูปภาพ 4 สถาปัตยกรรมเว็บเซอร์วิสบนแนวคิดของอาร์พีซีเซอร์วิส

หลังจากการพัฒนาระบบอาร์พีซีเพื่อให้บริการงานบนระบบอุปกรณ์ฝังตัวแล้ว หากต้องการเพิ่มความสามารถให้ระบบสามารถขยายการรองรับการเชื่อมต่อด้วยโปรโตคอลอื่นๆ หรือสร้างบริการใดๆ เพิ่มเติม ก็สามารถทำได้โดยทำการเพิ่มเติมโค้ดของส่วนต่าง ๆ นั้นเพิ่มเข้าไปในระบบ



รูปภาพ 5 มุมมองในการพัฒนาส่วนต่างเพิ่มเติมขึ้นในระบบ

กลไกการทำงานของส่วนต่างๆบนระบบอาร์พีซีเซิร์ฟเวอร์

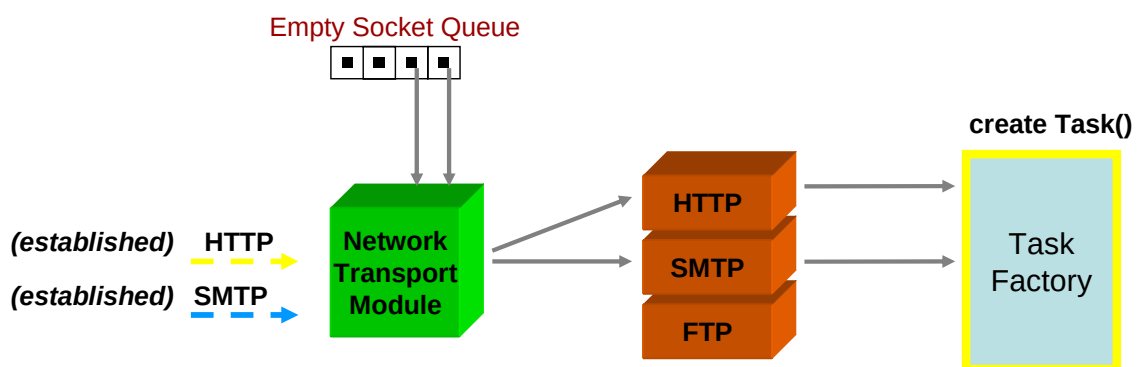
1. โพรเซสของผู้ให้บริการ หรือ เซิร์ฟเวอร์เดมอน (Server Daemon)
2. ส่วนจัดการโปรโตคอล หรือ โปรโตคอลแฮนเดิลเลอร์ (Protocol Handler)
3. ทาสก์และส่วนจัดการทาสก์ (Task & Task Manager)
4. บริการ และส่วนจัดการบริการ (Services & Service Manager)

โครงสร้างภายในของอาร์พีซีเซิร์ฟเวอร์ สามารถอธิบายส่วนประกอบและกลไกการทำงานได้ดังนี้

โพรเซสของผู้ให้บริการ หรือ เซิร์ฟเวอร์เดมอน (Server Daemon)

ส่วนของโพรเซสของผู้ให้บริการนี้ เป็นทาสก์ที่ทำงานอยู่ตลอดเวลา หรือที่เรียกว่า เดมอนโพรเซส (Daemon Process) ซึ่งเป็นทำหน้าที่เป็นโพรเซสที่คอยรับและทำหน้าที่เชื่อมการติดต่อในระดับเซสชันจากผู้ขอใช้บริการ โดยเมื่อผู้ขอใช้บริการส่งคำสั่งขอเชื่อมต่อ โพรเซสของผู้ให้บริการนี้จะสร้างช่องทางการติดต่อหรือซ็อกเก็ต (Socket) ขึ้นมาใหม่ แล้วบันทึกค่าต่างๆที่จำเป็นต่อการทำการติดต่อ แล้วเรียกให้ส่วนที่จัดการทาสก์ ทำการสร้างทาสก์ขึ้นมาใหม่ แล้วส่งต่อซ็อกเก็ตของการเชื่อมต่อครั้งนั้นๆ ไปเก็บไว้เป็นส่วนหนึ่งของทาสก์ จากนั้น โพรเซสนี้ ก็จะกลับไปรอรับฟังการเชื่อมต่อในเซสชันอื่นจากผู้ขอใช้บริการต่อไป

ในสถาปัตยกรรมของอาร์พีซีเซิร์ฟเวอร์ที่ได้ออกแบบมานั้น สามารถรองรับการทำงานของเซิร์ฟเวอร์เดมอนได้หลายๆ โพรเซสพร้อมกัน ซึ่งแต่ละโพรเซสนั้น จะรอรับการเชื่อมต่อจากพอร์ตการสื่อสารที่ต่างกันไป ตามหมายเลขพอร์ตที่รองรับโปรโตคอลต่างๆกันไป ทำให้สามารถรองรับการเชื่อมต่อได้พร้อมๆกัน ที่หลายๆโปรโตคอล



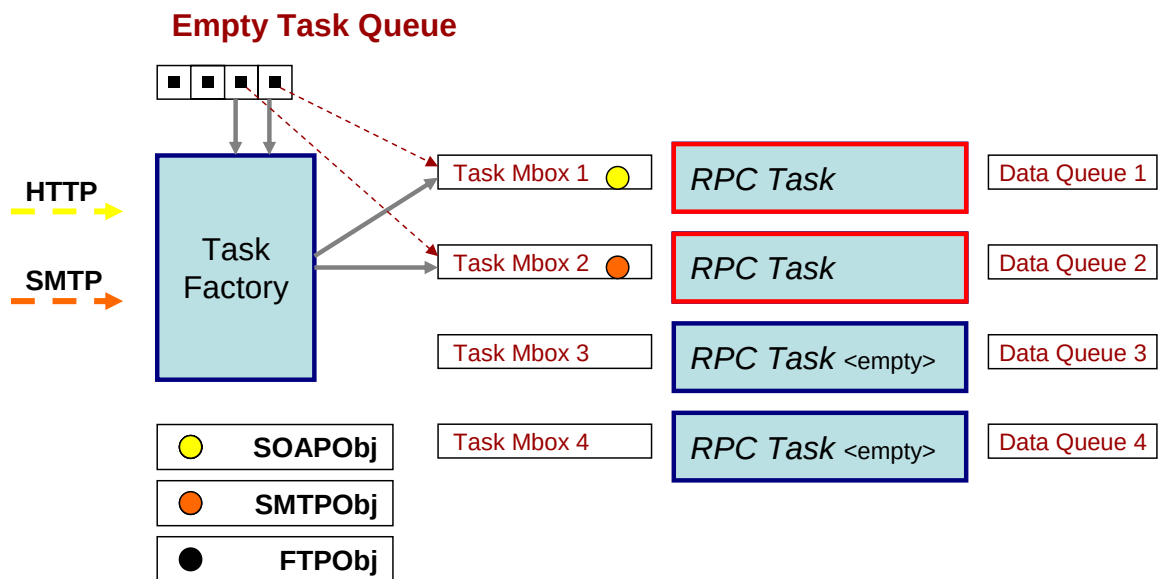
รูปภาพ 6 กลไกการรองรับรีเควสของเซิร์ฟเวอร์

ส่วนจัดการโปรโตคอล หรือ โปรโตคอลแฮนเดิลเลอร์ (Protocol Handler)

ส่วนจัดการโปรโตคอลนี้ เป็นส่วนที่แปลงเอกสารที่ส่งมาจากผู้ขอใช้บริการ ให้อยู่ในรูปแบบของข้อมูลที่ใช้ในการขอใช้บริการต่างๆ โดยมีรูปแบบตามแต่ละโปรโตคอลซึ่งเป็นมาตรฐานในการติดต่อ เช่น โซบแฮนเดิลเลอร์ ทำหน้าที่แปลงเอกสารโซบ แล้วนำข้อมูลไปขอเรียกใช้บริการต่อไป

ทาสก์และส่วนจัดการทาสก์ (Task & Task Manager)

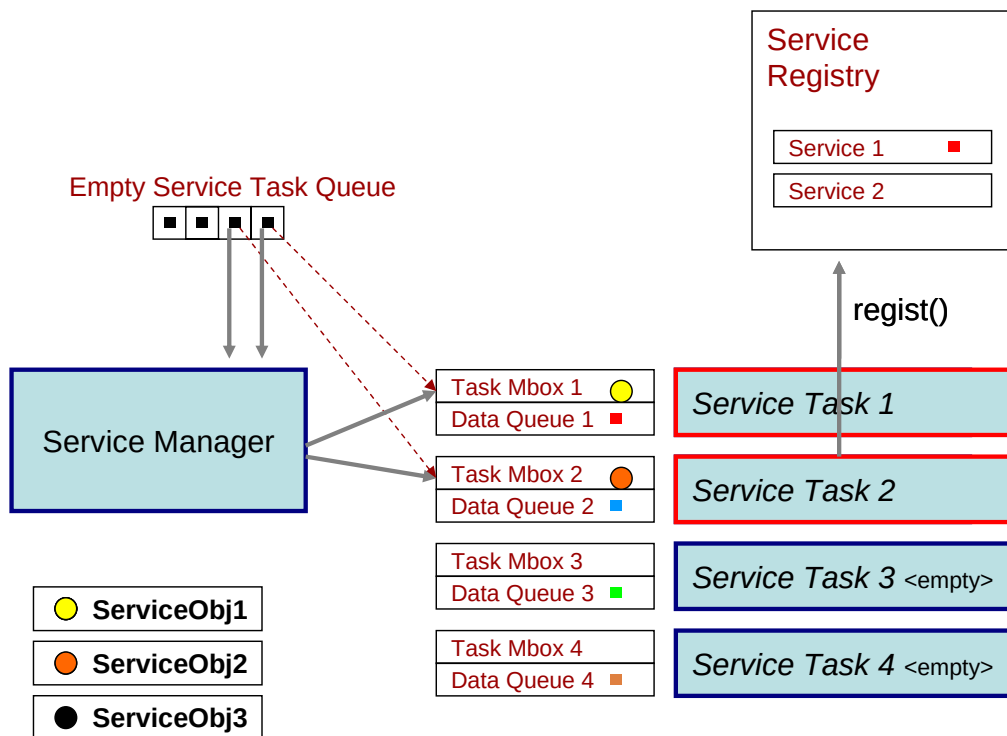
- **ทาสก์ (Task)** ที่เป็นรับผิดชอบในการดำเนินงานของรีเควส 1 รีเควส แล้วทำการดำเนินขั้นตอนตามลำดับที่กำหนดไว้ในแต่ละทาสก์ออบเจกต์ ตั้งแต่เริ่มต้น จนเสร็จกระบวนการการทำงานตามที่ผู้ขอใช้บริการทำการร้องขอ จากนั้นทาสก์นั้นก็จะว่าง แล้วกลับไปรอรับการสร้างเซอร์วิสต่อไป
- **ส่วนสร้างทาสก์ หรือ ทาสก์แฟคตอรี (Task Factory)** ทำหน้าที่ในการสร้างทาสก์ออบเจกต์เมื่อมีการร้องขอจากผู้ขอใช้บริการเข้ามา จากนั้นก็ทำการเลือกหาว่ามีทาสก์ใดที่อยู่ในสถานะที่ว่าง และพร้อมรับการทำงานอยู่ จากนั้นก็จะทำการส่งทาสก์ออบเจกต์ที่สร้างไว้แล้ว ไปให้กับทาสก์ที่ถูกเลือก เพื่อให้ทาสก์ๆนั้น รับผิดชอบในการดำเนินการตามขั้นตอนที่กำหนดอยู่ในทาสก์ออบเจกต์ที่ส่งให้ต่อไป



รูปภาพ 7 กลไกในการสร้างทาสก์ขึ้นเพื่อรองรับรีเควสตามแต่ละโปรโตคอล

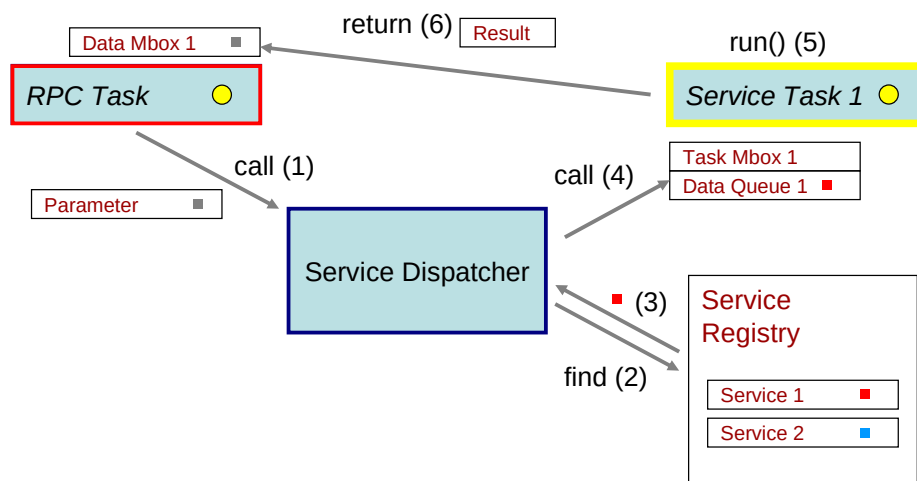
บริการ และส่วนจัดการบริการ (Services & Service Manager)

- **เซอร์วิสทาสก์ (Service Task)** เป็นทาสก์ที่ทำหน้าที่ดำเนินการกระบวนการของเซอร์วิสต่างๆ ที่มีในระบบ ให้รองรับการร้องขอจากผู้ขอใช้บริการ ซึ่งเซอร์วิสทาสก์นี้ จะถูกเรียกใช้โดยทาสก์ออบเจกต์ ที่ทำงานอยู่ในทาสก์ที่รองรับรีเควสชันนั้นๆ อยู่ แต่เซอร์วิสทาสก์จะถูกเรียกโดยผ่านเซอร์วิสดีสแพตเชอร์ อีกที แต่ในการส่งผลลัพธ์นั้น จะทำการส่งไปให้ยังทาสก์ที่ร้องขอโดยตรง ภายในเซอร์วิสทาสก์นั้นมีเซอร์วิสออบเจกต์อยู่ภายใน โดยเซอร์วิสออบเจกต์นี้เป็นตัวที่เก็บฟังก์ชันที่ใช้สำหรับการประมวลผล โดยเซอร์วิสหนึ่งๆนั้น จะถูกสร้างโดยเซอร์วิสเมนเจอร์ในตอนเริ่มแรกการทำงานของระบบ จากนั้นเซอร์วิสออบเจกต์ก็จะถูกส่งเข้ามายังเซอร์วิสทาสก์ซึ่งมีการเตรียมความพร้อมด้วยการสร้าง แมสเสจคิว (Message Queue) ที่จำเป็นในการรองรับการร้องขอจากเซอร์วิสดีสแพตเชอร์ แล้วทำการเริ่มต้นกระบวนการ จนไปตลอดเวลาขณะระบบทำงาน โดยขณะที่เซอร์วิสทาสก์ถูกเรียกใช้ผ่านการส่งข้อมูลมาที่แมสเสจคิว ของเซอร์วิสทาสก์นั้น แล้วเซอร์วิสทาสก์กำลังประมวลผลตามฟังก์ชันที่มีอยู่ หากมีทาสก์ใด ที่ต้องการขอบริการมาที่เซอร์วิสทาสก์เดียวกันนี้ ข้อมูลในการร้องขอบริการจะถูกนำเข้าไปในแมสเสจคิวเอาไว้ จนกว่าเซอร์วิสทาสก์นั้นจะทำงานเสร็จสิ้นงานที่ถูกร้องขอก่อนหน้านี้ แล้วจึงไปนำข้อมูลการร้องขอบริการจากแมสเสจคิวออกมาทำการประมวลเพื่อให้บริการต่อไป



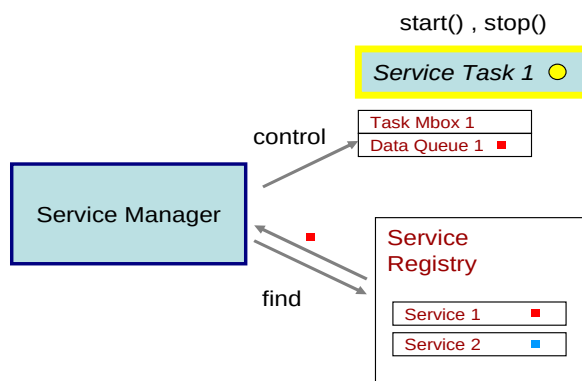
รูปภาพ 8 กลไกการสร้างเซอร์วิสขึ้นในระบบให้พร้อมทำงาน โดยมีหลายเซอร์วิสพร้อมกัน

- **เซอร์วิสดีสแพตчер (Service Dispatcher)** เป็นส่วนที่ทำหน้าที่แจกจ่ายการร้องขอใช้บริการไปยังเซอร์วิสทาสก์แต่ละอัน โดยจะตรวจหาว่าเซอร์วิสที่ทาสก์เรียกเข้ามานั้น มีอยู่ในระบบหรือไม่ ถ้ามี ก็จะไปค้นหาตำแหน่งของแมสเซจคิวของเซอร์วิสนั้นๆ เพื่อส่งข้อมูลการร้องขอบริการจากทาสก์ที่ร้องขอนั้นต่อไปยังเซอร์วิสทาสก์อีกที แต่หากเซอร์วิสที่ทาสก์ร้องขอนั้น ไม่มีอยู่ในรายการเซอร์วิสที่มีอยู่ในระบบแล้ว เซอร์วิสดีสแพตчерก็จะทำการส่งข้อมูลร้องขอเหล่านั้นไปยังบริการที่รองรับตอบบริการที่ผิดพลาด หรือ ฟอลต์เซอร์วิส (Fault Service) ซึ่งเป็นเซอร์วิสทาสก์หนึ่งในระบบที่คอยตอบความผิดพลาดกลับไปยังทาสก์ที่ร้องขอ เพื่อส่งข้อมูลความผิดพลาดในการเรียกใช้บริการกลับไปยังผู้ขอใช้บริการต่อไป



รูปภาพ 9 กลไกการขอใช้บริการเซอร์วิส

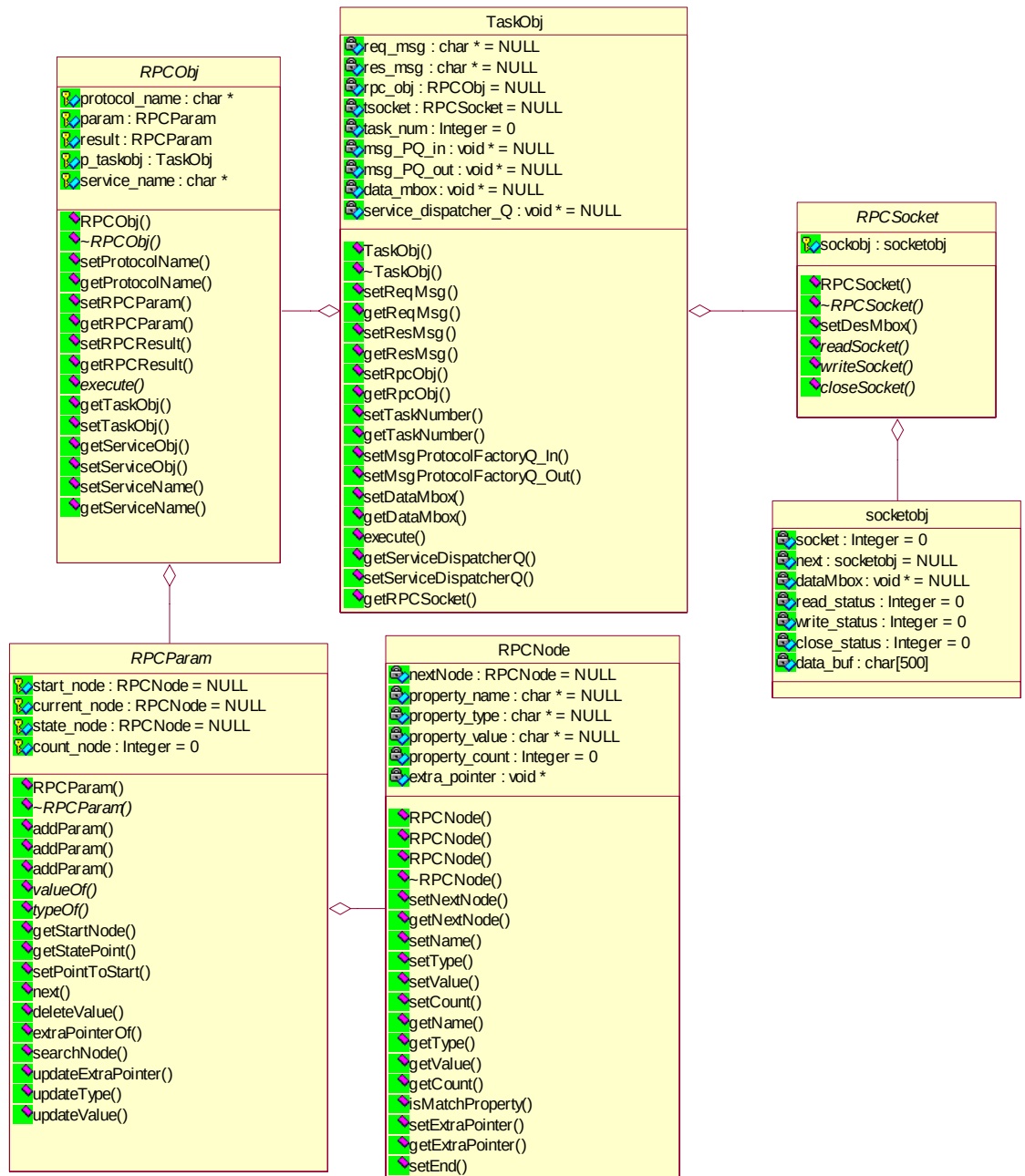
- **ผู้จัดการบริการ หรือ เซอร์วิสเมนเนเจอร์ (Service Manager)** เป็นส่วนที่ทำหน้าที่ในการสร้างเซอร์วิสออบเจกต์ที่มีอยู่ในระบบ แล้วส่งไปให้เซอร์วิสทาสก์ทำงานตามขั้นตอนที่มีอยู่ในออบเจกต์ หากเซอร์วิสใดมีสถานะให้หยุดบริการ เซอร์วิสเมนเนเจอร์ก็จะไม่สร้างเซอร์วิสออบเจกต์นั้นให้เซอร์วิสทาสก์ทำงาน แต่หากผู้ดูแลระบบทำการเปิดให้บริการนั้น เซอร์วิสทาสก์ก็จะสร้างเซอร์วิสออบเจกต์ขึ้นมา แล้วทำตามกระบวนการโดยส่งไปยังเซอร์วิสทาสก์ที่ว่างอยู่ เพื่อให้บริการนั้นเปิดใช้งานขึ้นมาอีกครั้ง



รูปภาพ 10 กลไกการทำงาน ในการควบคุมบริการในระบบ

รายละเอียดของโปรแกรม

1 คลาสไคอะแกรมของอาร์พีซีเซอร์วิส



รูปภาพ 11 คลาสไคอะแกรมของอาร์พีซีเซอร์วิส

1.1 คลาสทาสก์ออบเจกต์ (Task Object)

เป็นคลาสที่ทำหน้าที่ดำเนินการทำงานเพื่อรองรับการขอเรียกใช้บริการในครั้งนั้นๆ โดยในคลาสนี้จะเก็บรวบรวมข้อมูลที่ใช้เป็นในการติดต่อประสานงานกับทาสก์อื่นในระบบต่อไป

แอททริบิวต์ (Attribute)		
char*	req_msg	pointer ของ string ที่เป็นข้อมูลที่ได้รับมาจากผู้ร้องขอ
char*	res_msg	pointer ของ string ที่เป็นข้อมูลที่จะส่งกลับไปยังผู้ร้องขอ
RPCObj*	rpc_obj	pointer ของ RPC object ของทาสก์ออบเจกต์นี้
RPCSocket*	tsocket	pointer ของซ็อกเก็ตออบเจกต์ในการติดต่อกับผู้ใช้
int	task_num	หมายเลขของทาสก์ที่ทาสก์ออบเจกต์ทำงานอยู่
void*	msg_PQ_in	pointer ของ message queue ของ protocol in
void *	msg_PQ_out	pointer ของ message queue ของ protocol out
void *	data_mbox	pointer ของ message box สำหรับรับข้อมูลที่ส่งมาจากทาสก์อื่น
void *	service_dispatcher_Q	pointer ของ message queue ของ service dispatcher
คอนสตรัคเตอร์ (Constructor)		
TaskObj(RPCSocket *p_socket)		รับค่าพารามิเตอร์เป็น RPCSocket
ฟังก์ชัน (Function)		
void	setReqMsg	เซต pointer ของ message request
char *	getReqMsg	รีเทิร์น pointer ของ message request
void	setResMsg	เซต pointer ของ message response
char *	getResMsg	รีเทิร์น pointer ของ message response
void	setRPCObj	เซต pointer ของ rpc_obj
RPCObj*	getRPCObj	รีเทิร์น pointer ของ rpc_obj
void	setTaskNumber	เซตค่าหมายเลขของ task ที่ตัวเองทำงานอยู่
int	getTaskNumber	รีเทิร์นค่าหมายเลขของ task ที่ตัวเองทำงานอยู่
void	setMsgProtocolFactory_Q_In	เซต pointer ของ message queue ของ protocol in
void	setMsgProtocolFactory_Q_Out	เซต pointer ของ message queue ของ protocol out
void	setDataMbox	เซต pointer ของ data_mbox
void *	getDataMbox	รีเทิร์น pointer ของ data_mbox
void	setServiceDispatcherQ	เซต pointer ของ service_dispatcher_Q
void	setRPCSocket	เซต pointer ของ RPCSocket
void	execute	ดำเนินการสำหรับทาสก์ออบเจกต์นั้น

1.2 คลาสอาร์พีซีออบเจกต์ (RPC Object)

เป็นคลาสที่ทำหน้าที่ควบคุมการทำงานในส่วนของการติดต่อขอใช้บริการในระบบ รวมถึงรองรับการติดต่อในโปรโตคอลอื่นๆ โดยเมื่อนักพัฒนาต้องการอิมพลีเมนต์ก่อน โดยให้เขียนโปรแกรมเพิ่มเติมให้สามารถรองรับการทำงานในแต่ละโปรโตคอลที่ต้องการใช้ได้

แอททริบิวต์ (Attribute)		
char*	protocol_name	pointer ของ string ที่เป็นชื่อของโปรโตคอลที่ใช้อยู่
RPCParam*	param	pointer ของ object RPCParam ที่ใช้เป็นพารามิเตอร์ออบเจกต์
RPCParam*	result	pointer ของ object RPCParam ที่ใช้เป็นออบเจกต์ที่รีเทิร์นกลับมา
TaskObj*	p_taskobj	pointer ของ object TaskObj ที่ RPCObj ทำงานอยู่
char*	service_name	pointer ของ string ที่เป็นชื่อของบริการที่ต้องการเรียกใช้
คอนสตรัคเตอร์ (Constructor)		
RPCObj		
ฟังก์ชัน (Function)		
void	setProtocolName	เซ็ต pointer ของ protocol_name
char*	getProtocolName	รีเทิร์น pointer ของ protocol_name
void	setRPCParam	เซ็ต pointer ของ param
RPCParam*	getRPCParam	รีเทิร์น pointer ของ param
void	setRPCResult	เซ็ต pointer ของ result
RPCParam*	getRPCResult	รีเทิร์น pointer ของ result
void	setTaskObj	เซ็ต pointer ของ p_taskobj
TaskObj*	getTaskObj	รีเทิร์น pointer ของ p_taskobj
void	setServiceName	เซ็ต pointer ของ service_name
char*	getServiceName	รีเทิร์น pointer ของ service_name
void	execute	ดำเนินการในส่วนของการติดต่อเรียกขอบริการ

1.3 คลาสอาร์พีซีพารามิเตอร์ (RPC Param)

เป็นคลาสที่ใช้ในการเก็บข้อมูลและแลกเปลี่ยนข้อมูลระหว่างอาร์พีซีทาสก์ กับ เซอร์วิสทาสก์ ซึ่งจะช่วยให้การบันทึก ค้นหา และเรียกใช้ข้อมูลได้ง่ายขึ้น โดยภายในสร้างขึ้นโดยใช้ลิ่งคิลิส

แอททริบิวต์ (Attribute)		
RPCNode*	start_node	pointer ของ RPCNode object ที่เป็น node เริ่มแรกของ RPCParam
RPCNode*	current_node	pointer ของ RPCNode object ที่ชี้อยู่
RPCNode*	state_node	pointer ของ RPCNode object ที่ใช้ในการนับ state ใช้ในการค้นหา
int	count_node	จำนวนของ node ที่เก็บอยู่ในปัจจุบัน
คอนสตรัคเตอร์ (Constructor)		
RPCParam		
ฟังก์ชัน (Function)		
void	addParam	เพิ่ม node เข้าไป
void	addParam	เพิ่ม node เข้าไป
void	addParam	เพิ่ม node เข้าไป
char*	valueOf	ค่า value ของ node ที่ค้นหา
char*	typeOf	ค่า type ของ node ที่ค้นหา
RPCNode*	getStartNode	รีเทิร์น pointer ของ start_node
RPCNode*	getStatePoint	รีเทิร์น pointer ของ state_node
void	setPointToStart	ทำให้เริ่มชี้ที่ node เริ่มแรก
void	next	เลื่อนตำแหน่งของ node ที่ชี้อยู่
void	deleteValue	ลบ node ที่มีค่าตรงตามที่ต้องการ
void *	extraPointerOf	รีเทิร์น ค่า pointer พิเศษ
RPCNode*	searchNode	รีเทิร์น ค่า pointer ของ RPCNode ที่ได้จากการค้นหา
void	updateExtraPointer	บันทึกค่า pointer พิเศษใน RPCNode ที่ตรงตามที่ต้องการ
void	updateType	บันทึกค่า type ของ RPCNode ที่ได้จากการค้นหา
void	updateValue	บันทึกค่า value ของ RPCNode ที่ได้จากการค้นหาหรือสร้างขึ้นใหม่

1.4 คลาสอาร์พีซีโหนด (RPC Node)

เป็นคลาสที่เป็นหน่วยเก็บข้อมูลพื้นฐานของ RPCParam หรือเป็น node ของ linklist

แอททริบิวต์ (Attribute)		
RPCNode*	nextNode	pointer ไปที่ node ถัดไป
char*	property_name	pointer ของ name ของ node
char*	property_type	pointer ของ type ของ node
char*	property_value	pointer ของ value ของ node
int	property_count	ค่าจำนวนนับของ node
void *	extra_pointer	pointer พิเศษของ node
คอนสตรัคเตอร์ (Constructor)		
RPCNode		
ฟังก์ชัน (Function)		
void	setNextNode	เซต pointer ของ nextNode
RPCNode*	getNextNode	รีเทิร์น pointer ของ nextNode
void	setName	เซต property_name
void	setType	เซต property_type
void	setValue	เซต property_value
void	setCount	เซต property_count
char*	getName	รีเทิร์น property_name
char*	getType	รีเทิร์น property_type
char*	getValue	รีเทิร์น property_value
int	getCount	รีเทิร์น property_count
int	isMatchProperty	รีเทิร์นค่า 1 เมื่อพบ , รีเทิร์นค่า 0 เมื่อไม่พบ
void	setExtraPointer	เซต extra_pointer
void*	getExtraPointer	รีเทิร์น extra_pointer
void	setEnd	เซตให้เป็น node สุดท้าย ของ linklist

1.5 คลาสอาร์พีซีซ็อกเก็ต (RPC Socket)

เป็นคลาสที่จัดการการติดต่อสื่อสารข้อมูลผ่านแบบซ็อกเก็ต บนระบบ TCP/IP

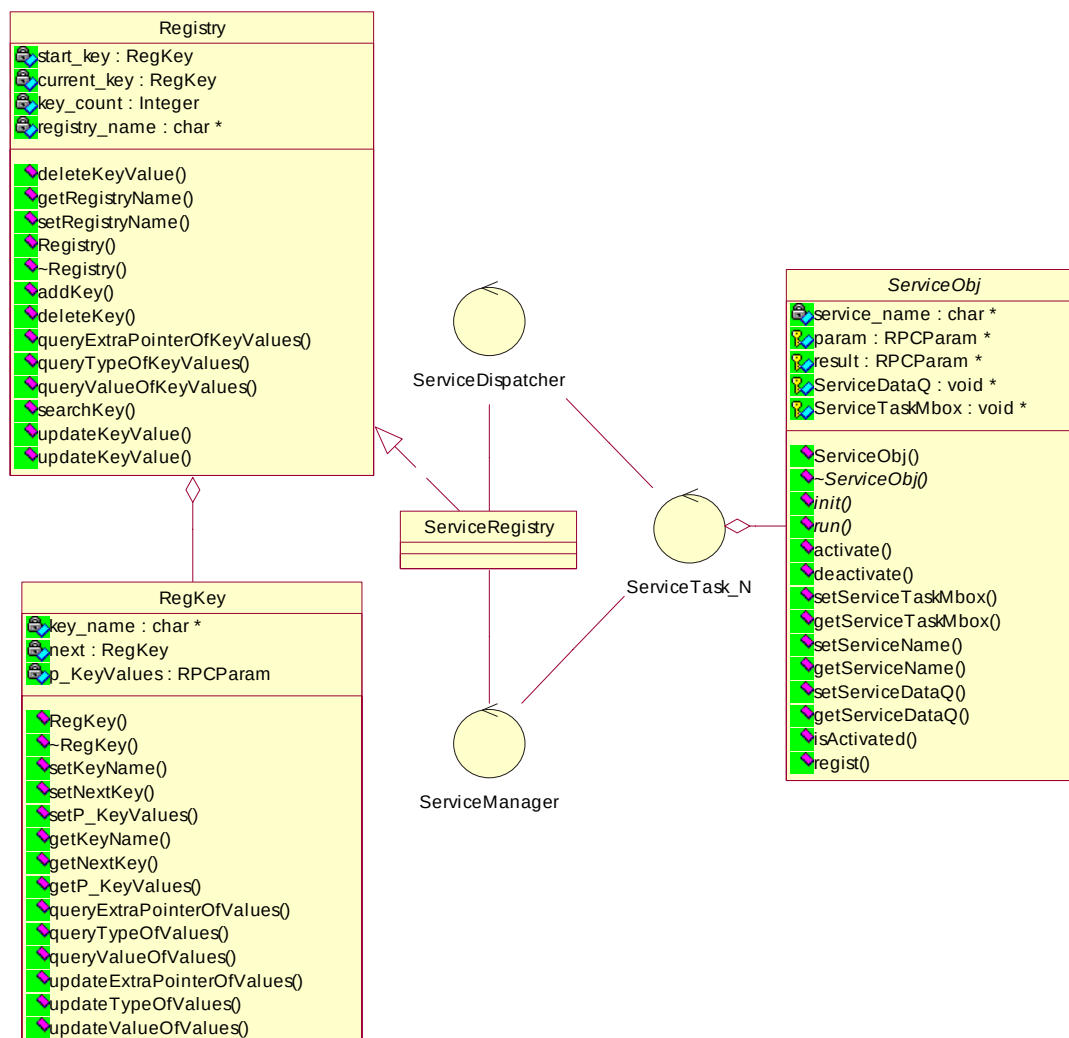
แอททริบิวต์ (Attribute)		
struct sockobj*	socketobj	เป็นโครงสร้างข้อมูลที่เก็บข้อมูลเกี่ยวกับการติดต่อผ่านซ็อกเก็ต
คอนสตรัคเตอร์ (Constructor)		
RPCSocket		
ฟังก์ชัน (Function)		
void	setDesMbox	เซตค่า pointer ของ message box ของซ็อกเก็ต
char*	readSocket	อ่านข้อมูลจากซ็อกเก็ต
int	writeSocket	เขียนข้อมูลลงซ็อกเก็ต
int	closeSocket	ปิดซ็อกเก็ต

1.6 สตรักซ็อกเก็ตออบเจกต์ (struct socketobj)

เป็นโครงสร้างข้อมูลที่เชื่อมต่อการแลกเปลี่ยนข้อมูลระหว่างเซิร์ฟเวอร์กับทาสก์เพื่อทำการเก็บข้อมูลการติดต่อผ่านซ็อกเก็ต

แอททริบิวต์ (Attribute)		
int	socket	หมายเลขพอร์ตของซ็อกเก็ตที่ทำการติดต่อ
struct socketobj *	next	socketobj ถัดไป
void*	dataMbox	pointer ของ message mbox ที่ใช้ในการแลกเปลี่ยนข้อมูลของซ็อกเก็ต
int	read_status	flag เพื่อการอ่าน
int	write_status	flag เพื่อการเขียน
int	close_status	flag เพื่อการปิดการเชื่อมต่อ
char[]	data_buf	buffer ข้อมูลที่จะอ่านหรือเขียน

2 คลาสไดอะแกรมของเซอร์วิสในระบบ



รูปภาพ 12 คลาสไดอะแกรมของเซอร์วิสในระบบ

2.1 คลาสเซอร์วิสออบเจกต์ (Service Object)

เป็นคลาสที่จะถูกอิมพลีเมนต์ไปสู่คลาสของบริการต่างๆที่จะพัฒนาขึ้นในระบบ โดยทำหน้าที่ในการประสานงานระหว่างทาสก์ที่ต้องการขอใช้บริการ รวมถึงดำเนินการประมวลผลภายในการร้องขอบริการนั้นๆ

แอททริบิวต์ (Attribute)		
char*	service_name	ชื่อของบริการนั้นๆ
RPCParam*	param	pointer ของ parameter object
RPCParam*	result	pointer ของ result object
void *	ServiceDataQ	pointer ของ message queue ของเซอร์วิสทาสก์สำหรับรอรับการร้องขอจากทาสก์อื่นๆ
void *	ServiceTaskMbox	pointer ของ message mbox ของเซอร์วิสทาสก์ สำหรับให้เซอร์วิสทาสก์เริ่มทำงาน
คอนสตรัคเตอร์ (Constructor)		
ServiceObj		
ฟังก์ชัน (Function)		
void	init	เป็นฟังก์ชันที่ต้องทำการอิมพลีเมนต์ก่อน โดยจะทำหน้าที่ในการติดตั้ง รวมถึงเซ็ตค่าพารามิเตอร์ลงในเซอร์วิสออบเจกต์ เพื่อเตรียมพร้อมในการประมวลผลบริการนั้นๆต่อไป
void	run	เป็นฟังก์ชันที่ดำเนินการประมวลผลโดยมีเซอร์วิสโค้ด ทำงานอยู่ภายใน
void	activate	ทำการเซ็ตสถานะที่พร้อมจะทำงาน
void	deactivate	ทำการเซ็ตสถานะที่หยุดการทำงาน
void	setServiceTaskMbox	เซ็ตค่า pointer ของ ServiceTaskMbox ไว้บอกตำแหน่งของฟังก์ชัน
void *	getServiceTaskMbox	รีเทิร์นค่า pointer ของ ServiceTaskMbox
void	setServiceName	เซ็ต pointer ของ service_name
char *	getServiceName	รีเทิร์น pointer ของ string service_name
void	setServiceDataQ	เซ็ต pointer message queue ที่ไว้รับค่าการร้องขอใช้บริการจากทาสก์อื่น
void *	getServiceDataQ	รีเทิร์น pointer ของ ServiceDataQ
int	isActivated	รีเทิร์นสถานะการพร้อมใช้งานของระบบ
void	regist	จดทะเบียนรายละเอียดของบริการกับเซอร์วิสรีจิสทรี

2.2 คลาสรีจิสทรี (Registry)

เป็นคลาสยูทิลิตี้ ที่ช่วยจัดเก็บกลุ่มของข้อมูลให้มีระบบและง่ายต่อการจัดการ

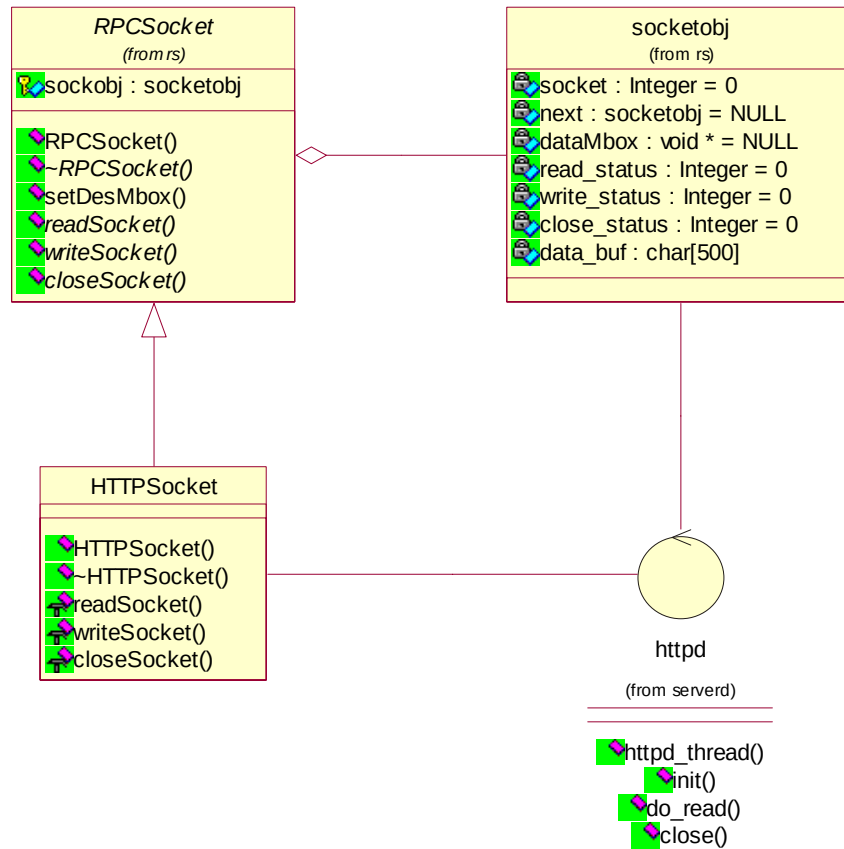
แอททริบิวต์ (Attribute)		
RegKey*	start_key	pointer ของออบเจกต์ RegKey เริ่มต้น
RegKey*	current_key	pointer ของออบเจกต์ RegKey ที่ชี้เพื่อการค้นหา
int	key_count	จำนวน key ที่มีอยู่ใน registry นั้น
char *	registry_name	ชื่อของ registry
คอนสตรัคเตอร์ (Constructor)		
Registry		
ฟังก์ชัน (Function)		
void	deleteKeyValue	ลบค่า value ของ key ที่ตรงกับที่ต้องการจะลบ
char*	getRegistryName	รีเทิร์น pointer ของ string ที่เป็นชื่อของ registry
void	setRegistryName	เซต pointer ของ string ที่เป็นชื่อของ registry
RegKey*	addKey	เพิ่ม key เข้าไป
void	deleteKey	ลบ key ที่มีชื่อตรงกับที่ต้องการจะลบ
void*	queryExtraPointerOfKeyValues	รีเทิร์น pointer พิเศษที่เก็บไว้ใน value ของ key ที่ต้องการค้นหา
char*	queryTypeOfKeyValues	รีเทิร์น pointer ของ string ที่เป็น type ของ value ที่ต้องการหา
char*	queryValueOfKeyValues	รีเทิร์น pointer ของ string ที่เป็น value ของ value ที่ต้องการหา
RegKey*	searchKey	รีเทิร์น key ที่ต้องการค้นหา
void	updateKeyValue	เซต ค่า value name , type , value ของ key ที่ตรงตามที่กำหนด
void	updateKey	เซต ค่า value name , type , value และ pointer พิเศษของ key ที่ตรงตามที่กำหนด

2.3 คลาสรีจิสทรีคีย์ (Registry Key)

เป็นคลาสที่เก็บข้อมูลหน่วยหนึ่ง ของคลาสรีจิสทรี ซึ่งทำหน้าที่เป็น key object

แอททริบิวต์ (Attribute)		
char*	key_name	pointer ของ string ที่เป็นชื่อของ key
คอนสตรัคเตอร์ (Constructor)		
RegKey		
ฟังก์ชัน (Function)		
void	setKeyName	เซต pointer ของ string ที่เป็นชื่อของ key
void	setNextKey	เซต pointer ของ RegKey ถัดไป
void	setP_KeyValues	เซต poionter ของ value ซึ่งเป็น RPCParameter
char*	getKeyName	รีเทิร์น pointer ของ string ที่เป็นชื่อของ key
RegKey*	getNextKey	รีเทิร์น pointer ของ RegKey ถัดไป
RPCParam*	getP_KeyValues	รีเทิร์น pointer ของ value ซึ่งเป็น RPCParameter
void *	queryExtraPointerOfValues	รีเทิร์น pointer พิเศษ ของ value ที่ตรงกับที่ค้นหา
char*	queryTypeOfValues	รีเทิร์น pointer ของ string ที่เป็น type ของ value m ต้องการค้นหา
char*	queryValueOfValues	รีเทิร์น pointer ของ string ที่เป็น value ของ value ที่ ต้องการค้นหา
void	updateExtraPointerOfValues	เซต pointer พิเศษ ที่อยู่ใน value ที่ต้องการจัดเก็บ
void	updateTypeOfValues	เซต pointer ของ string ที่เป็น type ของ value ที่ ต้องการจัดเก็บ
void	updateValueOfValues	เซต pointer ของ string ที่เป็น value ของ value m ต้องการจัดเก็บ

3 คลาสไดอะแกรมของซ็อกเก็ตการเชื่อมต่อ



รูปภาพ 13 คลาสไดอะแกรมของซ็อกเก็ตการเชื่อมต่อ

3.1 คลาสเอชทีทีพีซ็อกเก็ต

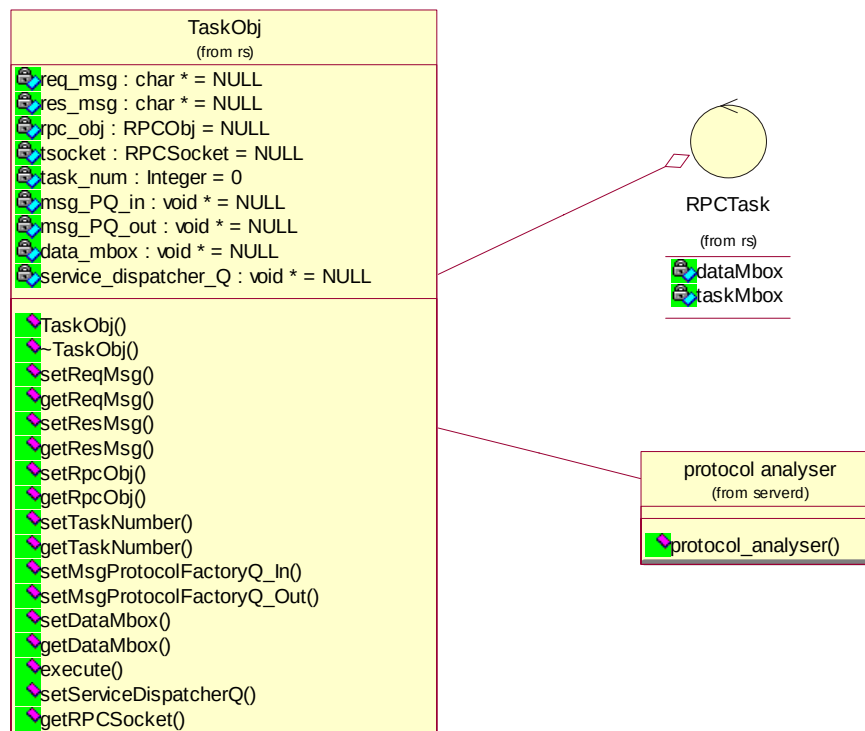
เป็นคลาสสืบทอดมาจากคลาสอาร์พีซีซ็อกเก็ต เพื่อทำหน้าที่เฉพาะในการติดต่อกับผู้ร้องขอบริการด้วยโปรโตคอลเอชทีทีพี (HTTP)

แอททริบิวต์ (Attribute)		
struct socketobj *	socketobj	(สืบทอดมาจาก RPCSocket)
คอนสตรัคเตอร์ (Constructor)		
HTTPSocket		
ฟังก์ชัน (Function)		
char*	readSocket	(สืบทอดมาจาก RPCSocket)
void	writeSocket	(สืบทอดมาจาก RPCSocket)
void	closeSocket	(สืบทอดมาจาก RPCSocket)

3.2 กลุ่มฟังก์ชันเอชทีทีพีเซิร์ฟเวอร์

ฟังก์ชัน (Function)		
void	httpd_thread	เป็นส่วนหลักของเอชทีทีพีเซิร์ฟเวอร์
void	init	เป็นส่วนของการเริ่มต้นและตั้งค่าเอชทีทีพีเซิร์ฟเวอร์
void	do_read	ทำหน้าที่ในการอ่านข้อมูลมาจากที่ซีพียูสแต็กมาเก็บไว้ในบัฟเฟอร์
void	close	ปิดการทำงานของเอชทีทีพีเซิร์ฟเวอร์

4 คลาสไลอะแกรมของทาสก์แฟคตอรี



4.1 ฟังก์ชันอาร์พีซีทาสก์ (RPC Task function)

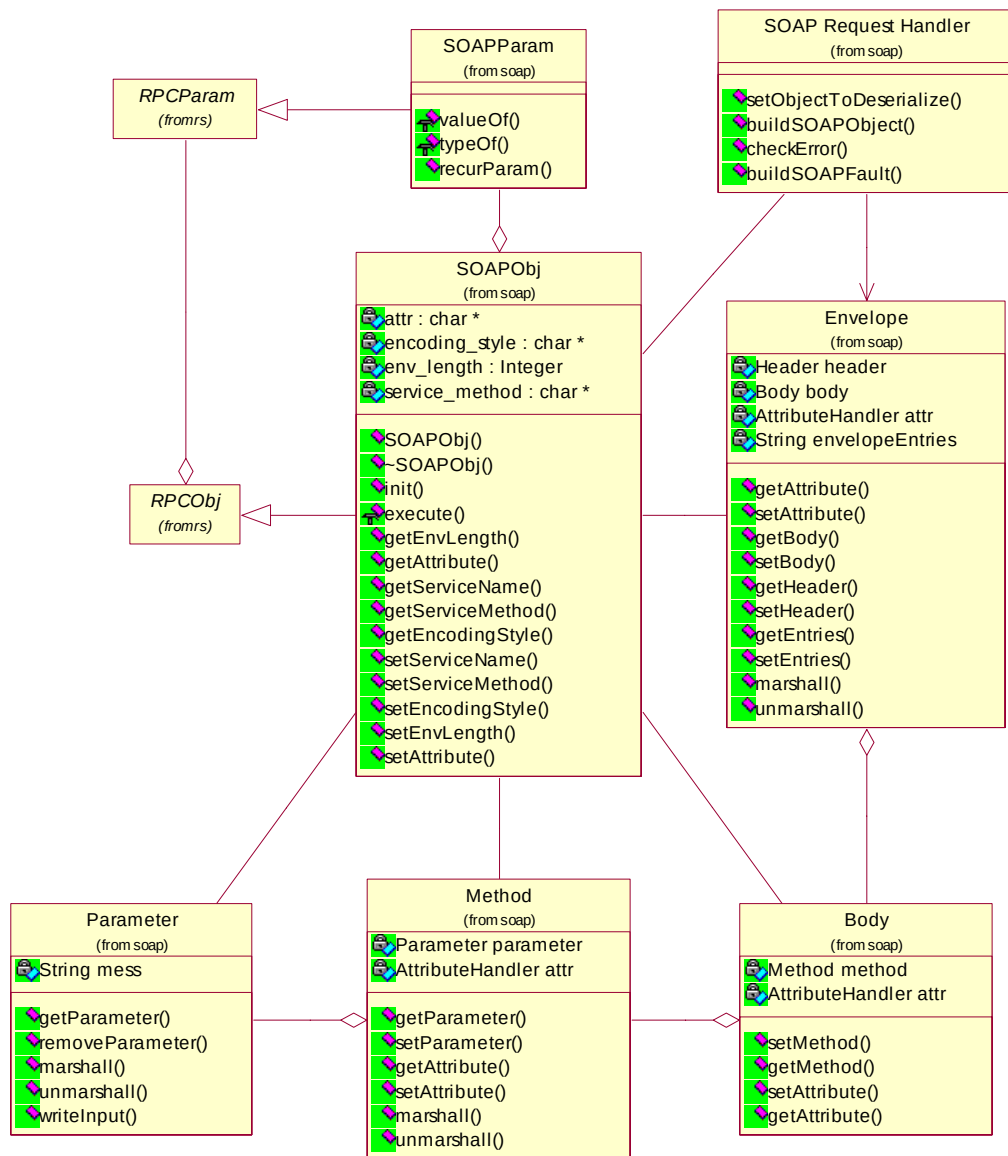
ฟังก์ชัน (Function)		
void	RPCTask	เป็นฟังก์ชันที่ทำหน้าที่เป็นทาสก์ สำหรับรองรับการทำงานรองรับการร้องขอบริการ โดยภายในจะมีการ Task Object อยู่ภายใน

4.2 ฟังก์ชันวิเคราะห์โปรโตคอล (Protocol Analyzer)

ฟังก์ชัน (Function)		
void	protocol_analyzer	เป็นฟังก์ชันที่ทำหน้าที่วิเคราะห์ว่า ซอกเก็ตเกิดการติดต่อที่จะสร้างขึ้นนั้น อยู่ใน port ไດ และต้องสร้าง RPC Object ตัวใดขึ้นรองรับ

5 คลาสไดอะแกรมของโซบพาร์เซอร์และเอนโค้ดเดอร์

เป็นกลุ่มของคลาสที่ทำหน้าที่ในการอ่านเอกสารโซบ โดยรายละเอียดของคลาสเหล่านี้ จะอยู่ในเอกสารคู่มือโปรแกรมเมอร์ โครงการ การพัฒนาเว็บเซอร์วิสบนระบบฝังตัว ปีการศึกษา 2545 ยกเว้นส่วนที่ได้ทำเพิ่มเติมขึ้นในโครงการนี้ ได้แก่ โซบออบเจกต์ (SOAP Object) ที่ทำหน้าที่เป็นคลาสของออบเจกต์ที่ทำหน้าที่ประสาน และรวบรวมข้อมูลที่ใช้ในการอ่าน และเขียนเอกสารโซบ



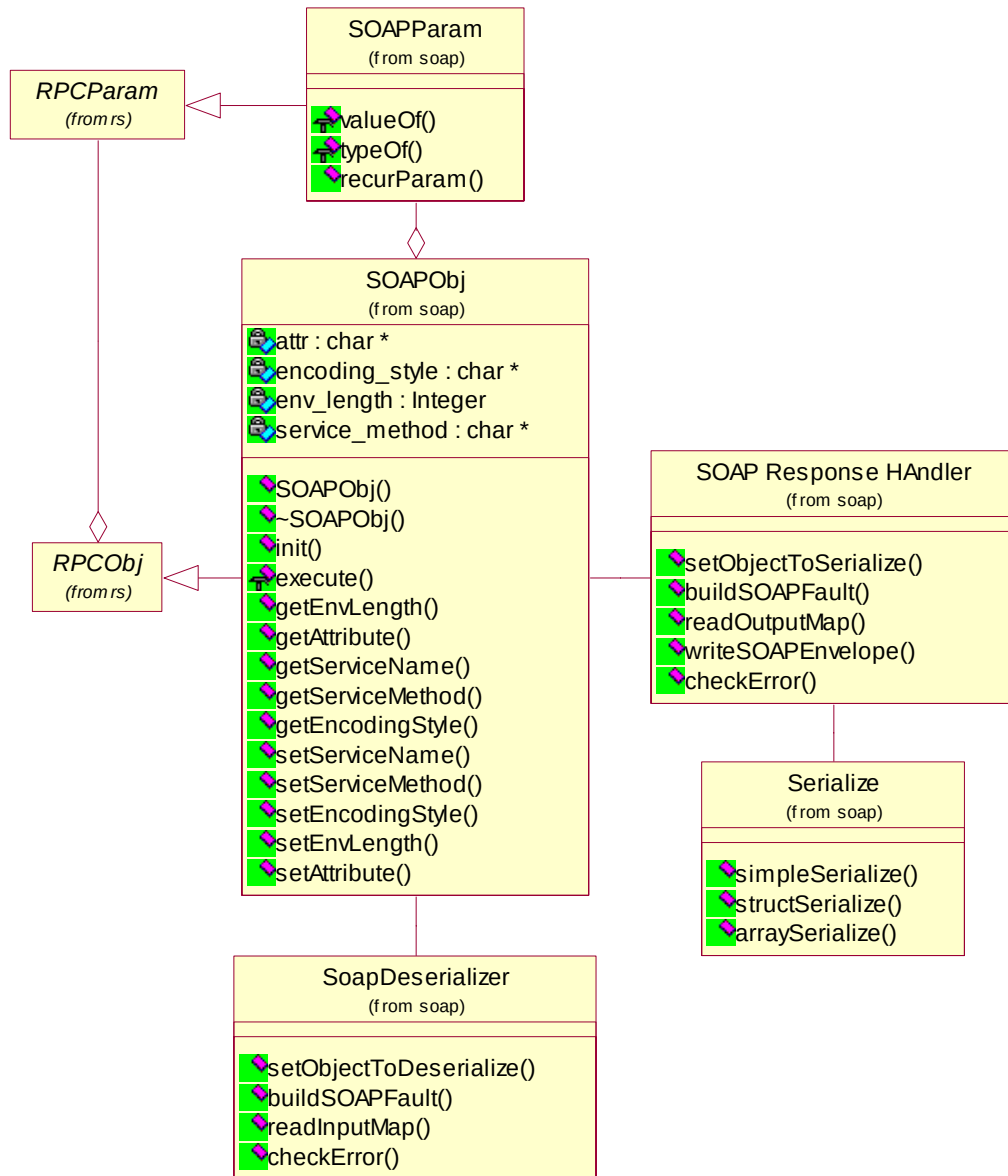
รูปภาพ 14 คลาสไดอะแกรมของโซบพาร์เซอร์และเอนโค้ดเดอร์

5.1 คลาสโซบอบเจ็คต์

แอททริบิวต์ (Attribute)		
char *	attr	แอททริบิวต์ของเอกสาร โซบ
char *	encoding_style	encoding style ของเอกสาร โซบ
int	env_length	ความยาวของเอกสาร โซบ
char *	service_method	เซอร์วิสเมธอดของการเรียกใช้บริการ
คอนสตรัคเตอร์ (Constructor)		
SOAPObj		
ฟังก์ชัน (Function)		
void	init	อิมพลิเมนต์จากคลาส RPC Object สำหรับการเซ็ตค่าสำหรับการทำ RPC ในรูปแบบของโซบ
int	execute	อิมพลิเมนต์จากคลาส RPC Object สำหรับการทำการประมวลผลของการทำ RPC ในรูปแบบของโซบ
int	getEnvLength	รีเทิร์นค่าความยาวของเอกสาร โซบ
char *	getAttribute	รีเทิร์น string ที่เป็นแอททริบิวต์
char *	getServiceName	รีเทิร์น string ที่เป็นชื่อของ service ที่ต้องการเรียก
char *	getServiceMethod	รีเทิร์น string ที่เป็นชื่อของ method ที่ต้องการเรียก
void	setServiceName	เซ็ต string ที่เป็นชื่อของ service ที่ต้องการเรียก
void	setServiceMethod	เซ็ต string ที่เป็นชื่อของ method ที่ต้องการเรียก
void	setEncodingStyle	เซ็ต string ที่เป็น encoding style
void	setEnvLength	เซ็ตค่าความยาวของเอกสาร โซบ
void	setAttribute	เซ็ต string ที่เป็นแอททริบิวต์

6 คลาสไดอะแกรมของโซบเอนเวลลัฟและซีเรียไลซ์

เป็นกลุ่มของคลาสที่ทำหน้าที่ในการอ่านเอกสารโซบ โดยรายละเอียดของคลาสเหล่านี้ จะอยู่ในเอกสารคู่มือโปรแกรมเมอร์ โครงการ การพัฒนาเว็บเซอร์วิสบนระบบฝังตัว ปีการศึกษา 2545



รูปภาพ 15 คลาสไดอะแกรมของโซบเอนเวลลัฟและซีเรียไลซ์